



Dokumentacija implementacije

MP3 Music Player

Adnan Dervišević

Ahmed Čorhušić

Adnan Ćorović

Sara Dautbegović



1 | Uvod

Za implementaciju smo koristili dva Raspberry Pi Pico mikrokontrolera. Jedan mikrokontroler je zadužen za izgled displeja, a drugi za reprodukciju zvuka i učitavanje pjesama sa SD kartice. Njihovu komunikaciju smo uspostavili putem UART-a.

2 | Raspberry Pi Pico 1

Prvi Raspberry Pi Pico je zadužen za općenitu interakciju korisnika sa sistemom, tj. upravljanje displejem koristeći LVGL biblioteku (https://github.com/lvgl/lv_micropython), te primanje ulaza preko nekog od 4 tastera, te rotacijskog enkodera. Ovaj Pico komunicira sa drugim preko UARTA.

2.1 | Pokretanje prvog Raspberry Pi Pico-a

Prilikom pokretanja, preko UARTA šalje poruku "Hello" i kratko postavi vrijednost na 1 na pinu kojim se drugom Raspberry-ju signalizira da postoji poruka na UART-u. To ponavlja dok god drugi pico ne odgovori sa "start", nakon čega počinje pristizati lista pjesama i njihovih dužina, koje razdvajamo i spašavamo u niz dvojki. Nakon poruke "end", izlazi se iz petlje, a zatim se postavlja korisnički interfejs. Inicijaliziraju se 3 tile-a, koji odgovaraju respektivno modovima "Welcome", "List" i "Song". Postavljaju se i odgovarajući widgeti i stilovi na svaki od njih. Učitavamo sliku i postavljamo na tile 1, a na tile 2 se postavlja lista u koju se dodaje 7 button-a sa nazivima pjesama. Istovremeno se kreira i niz referenci na te button-e, da bismo ih kasnije mogli mijenjati. Postavljaju se interrupti na sve ulaze i inicijaliziraju se ostale potrebne varijable.

```
1  import lvgl as lv
2  from machine import Pin, Timer, SPI, UART
3  import ili9xxx
4  import gc
5  import uos
6  import time
7  import _thread
8
9  DEBOUNCE_TIME = 200 # u ms
10 last_button_press_time = 0
11
12 uart1 = UART(0, baudrate=9600, tx=Pin(12), rx=Pin(13))
13
14 spi = SPI(0, baudrate=50_000_000, sck=Pin(18), mosi=Pin(19), miso=Pin(16))
15
16 drv = ili9xxx.Ili9341(spi=spi, dc=15, cs=17, rst=20)
17
18 gc.collect()
19 songList = [] # Lista ucitanih pjesama
20
21 trigg = Pin(11,Pin.OUT)
22 trigPlay = Pin(15,Pin.OUT)
```

2.2 | Main

Čitav kod se izvršava u jednoj beskonačnoj petlji koja konstantno prima poruke od Raspberry-ja

2. Dokle god je ta poruka “start”, program se izvršava i Music Player pruža usluge korisniku. Nakon što Raspberry 2 pošalje poruku “end”, rad Player-a se obustavlja. U toj petlji su se i učitala

imena pjesama koje je putem UART komunikacije poslao Raspberry PiPico2.

```
44 while not gasiSve:
45     uart1.write('Hello')
46     time.sleep(2)
47
48     trigg.value(1)
49     time.sleep(1)
50     trigg.value(0)
51
52
53     if uart1.any():
54         message = uart1.read().decode('utf-8')
55         if message == "start":
56             while True:
57                 if uart1.any():
58                     message = uart1.read().decode('utf-8')
59                     if message == "end":
60                         gasiSve = True
61                         break
62                     songList.append(message.split(", "))
63                     print(songList[-1][0], " - ", songList[-1][1])
64                     time.sleep(0.2)
65             break
66     time.sleep(1)
```

Na četiri korištena tastera inicijalizirani su prekidi.

```
499 # Inicijalizacija prekida na 4 koristena tastera
500 btn1 = Pin(0, Pin.IN)
501 btn1.irq(trigger=Pin.IRQ_FALLING, handler=changeToList)
502
503 btn2 = Pin(1, Pin.IN)
504 btn2.irq(trigger=Pin.IRQ_FALLING, handler=playSong)
505
506 btn3 = Pin(2, Pin.IN)
507 btn3.irq(trigger=Pin.IRQ_FALLING, handler=prevSong)
508
509 btn4 = Pin(3, Pin.IN)
510 btn4.irq(trigger=Pin.IRQ_FALLING, handler=nextSong)
```

2.2.1 | Pokretanje i zaustavljanje pjesama

Pri pritisku na Taster_2 se poziva playSong, koja će postaviti tile 3 prema pjesmi, te ukoliko smo tek promijenili na tu pjesmu, šalje se zahtjev Pico-u 2 da pusti pjesmu sa trenutnim indeksom. Zahtjev se šalje preko UART-a porukom "Play " nakon koje ide indeks trenutne pjesme. Uz to se signalizira preko odgovarajućeg pina da ima poruka na UART-u. Također resetuju određene

varijable, te pokreće timer koji ažurira time bar i prati koliko je pjesme prošlo. Ukoliko je pjesma pokrenuta, a pritisnut je Taster_2, poslat će se zahtjev za pauziranje porukom "Pause" i signalizacijom na odgovarajući pin, te će se timer deinicijalizirati. Ukoliko je pjesma pauzirana, šalje se zahtjev za njenim ponovnim pokretanjem porukom "Resume", pri čemo opet adekvatno signaliziramo, te pokrenemo timer. Ukoliko se iz "Song" moda vratimo u "List" mod ili ukoliko se promijeni pjesma, pozvat će se zahtjev za zaustavljanjem pjesme porukom "Stop", te odgovarajućim signaliziranjem.

```
105 def playSong(pin):
106     global songList, sviraPjesma, currentSongIndex, mode, pauzirano
107     global timer, prosloPjesme
108     if mode == 'Welcome':
109         return
110     changeToSong(pin)
111     gc.collect()
112     if not sviraPjesma:
113         uart1.write('Play ' + str(currentSongIndex))
114         prosloPjesme = 0
115         sviraPjesma = True
116         pauzirano = False
117         time.sleep(2)
118         timer.init(callback = update_bar, period = 1000, mode = Timer.PERIODIC)
119         trigPlay.value(1)
120         time.sleep(1)
121         trigPlay.value(0)
122         print("Pustam pjesmu\n")
123     elif pauzirano:
124         uart1.write('Resume')
125         pauzirano = False
126         time.sleep(2)
127         timer.init(callback = update_bar, period = 1000, mode = Timer.PERIODIC)
128         trigPlay.value(1)
129         time.sleep(1)
130         trigPlay.value(0)
131         print("Pustam pjesmu\n")
132     else:
133         uart1.write('Pause')
134         pauzirano = True
135         time.sleep(2)
136         timer.deinit()
137         trigg.value(1)
138         time.sleep(1)
139         trigg.value(0)
```

2.2.2 Listanje i odabir pjesama

Za skrolanje liste se koristi rotacijski enkoder ili tasteri Taster_3 ili Taster_4. Oni pozivaju funkcije nextSong ili prevSong koje kada je trenutni mod "List" zapravo mijenjaju indeks trenutne pjesme, pri čemu prije promjene indeksa vrata obični stil na button, a nakon promjene postavke selektovani stil na odgovarajući button. Kako se koristi samo 7 button-a, ukoliko treba izvršiti skrol na pjesmu



koja se ne vidi na ekranu, pozvat će se `pomjeriDolje` ili `pomjeriGore`, koje će ažurirati varijable indeksa prve i zadnje prikazane pjesme, te promijeniti tekstove button-a tako da se izvrši pomjeranje ka dolje ili gore. Ukoliko smo došli na kraj liste pjesama, a želimo nastaviti u istom smjeru, pozvat će se `postaviNaPocetak` koja će postaviti button-e na prvih 7 pjesama, te selektovati prvu. Za obrnutu situaciju, kada smo na prvoj pjesmi, a želimo nazad, poziva se `postaviNaKraj` koja postavlja button-e na zadnjih 7 pjesama i selektuje posljednju. Obje funkcije ažuriraju indekse prve i zadnje prikazane pjesme.

```
207 def nextSong(pin):
208     global currentSongIndex, songList, tipke, mode, DEBOUNCE_TIME, last_button_press_time, prva_prikazana_pjesma, zadnja_prikazana_pjesma,
209     | pauzirano, sviraPjesma, timer
210     current_time = time.ticks_ms()
211     if time.ticks_diff(current_time, last_button_press_time) < DEBOUNCE_TIME:
212         return
213     last_button_press_time = current_time
214
215     if mode == 'List':
216         tipke[currentSongIndex - prva_prikazana_pjesma].replace_style(selectedTrackStyle, trackStyle, 0)
217         if currentSongIndex + 1 == len(songList):
218             postaviNaPocetak()
219         elif currentSongIndex == zadnja_prikazana_pjesma and zadnja_prikazana_pjesma + 1 < len(songList):
220             pomjeriGore()
221         else:
222             currentSongIndex = currentSongIndex + 1
223         tipke[currentSongIndex - prva_prikazana_pjesma].replace_style(trackStyle, selectedTrackStyle, 0)
224         gc.collect()
225     elif mode == 'Song':
226         if sviraPjesma and not pauzirano:
227             stopSong()
228         else:
229             sviraPjesma = False
230             pauzirano = False
231             currentSongIndex = (currentSongIndex + 1) % len(songList)
232             changeToSong(pin)
233             playSong(pin)
234             pass
```

```
237 def prevSong(pin):
238     global currentSongIndex, songList, mode, DEBOUNCE_TIME, last_button_press_time, prva_prikazana_pjesma, zadnja_prikazana_pjesma, sviraPjesma,
239     | pauzirano, sviraPjesma, timer
240     tmp = last_button_press_time
241     current_time = time.ticks_ms()
242     last_button_press_time = current_time
243     if time.ticks_diff(current_time, tmp) < DEBOUNCE_TIME:
244         return
245
246     if mode == 'List':
247         tipke[currentSongIndex - prva_prikazana_pjesma].replace_style(selectedTrackStyle, trackStyle, 0)
248
249         if currentSongIndex == 0:
250             postaviNaKraj()
251         elif currentSongIndex == prva_prikazana_pjesma and prva_prikazana_pjesma > 0:
252             pomjeriDolje()
253         else:
254             currentSongIndex = currentSongIndex - 1
255
256         tipke[currentSongIndex - prva_prikazana_pjesma].replace_style(trackStyle, selectedTrackStyle, 0)
257         gc.collect()
258     elif mode == 'Song':
259         if sviraPjesma and not pauzirano:
260             stopSong()
261         else:
262             sviraPjesma = False
263             pauzirano = False
264             currentSongIndex = currentSongIndex - 1
265             if currentSongIndex < 0:
266                 currentSongIndex = len(songList) - 1
267             changeToSong(pin)
268             playSong(pin)
```

2.2.3 Prebacivanje pjesama

Ukoliko smo u modu "Song", a okrene se enkoder ili se pritisne Taster_3 ili Taster_4, zaustavit će se pjesma, te će se izvršiti odgovarajuća promjena indeksa trenutne pjesme, nakon čega se pozivaju funkcije da u skladu s tome izmijene tekstove labela na tile 3, te playSong, da bi se pjesma pokrenula. Također, do ovoga dolazi kada timer za ažuriranje time bara uvidi da je pjesma završila.

```
154 def changeToSong(pin):
155     global tv, tile3, label1, currentSongIndex, mode, songList, prva_prikazana_pjesma,
156     selectedTrackStyle, trackStyle, trajanjePjesme, prosloPjesme, dur_labelMax
157     if mode != 'Welcome':
158         if mode == 'List':
159             tipke[currentSongIndex - prva_prikazana_pjesma].replace_style(selectedTrackStyle, trackStyle, 0)
160             mode = 'Song'
161             vr = songList[currentSongIndex][1].split(":")
162             trajanjePjesme = int(vr[0]) * 60 + int(vr[1])
163
164             label1.set_text(songList[currentSongIndex][0])
165             dur_labelMax.set_text(songList[currentSongIndex][1])
166             tv.set_tile(tile3, lv.ANIM.ON)
```

2.3 | Korisnički interfejs

Za izgled korisničkog interfejsa koristili smo LVGL biblioteku. Koristili smo objekte tipa Tile, kako bismo realizirali tri stranice.

Na Tile-u 1 nalazi se početna stranica displeja sa importovanim logom.

```
288 # Kreiranje 2x2 tileview-a i omogućavanje skrolanja u obliku slova "L".
289 tv = lv.tileview(lv.screen_active())
290
291 tvStyle = lv.style_t()
292 tvStyle.init()
293 tvStyle.set_border_width(0)
294
295 tile1 = tv.add_tile(0, 0, lv.DIR.BOTTOM)

324 # Učitavanje slike u 'Welcome' modu
325 with open('/etf_logo.png', 'rb') as f:
326     png_data = f.read()
327
328 png_image_dsc = lv.image_dsc_t({
329     'data_size': len(png_data),
330     'data': png_data
331 })
332 # time.sleep(1)
333
334
335 # Kreiraj sliku korištenjem dekodera
336 image1 = lv.image(tile1)
337 image1.set_src(png_image_dsc)
338 image1.center()
339 image1.set_scale(500)
```

Na Tile-u 2 prikazana je lista pjesama. Tile 2 je inicijaliziran kao i Tile 1. Ovdje ćemo prikazati primjer inicijalizacije stila u LVGL biblioteci. Stil trackStyle se dodijelio listi button-a, a stil selectedTrackStyle će se dodijeliti onom button-u u listi koji je trenutno izabran.

U for petlji puni se varijabla list tipa button. Na labelama na button-ima ispisuju se tekstovi pjesama, dodaje se stil. Nizu tipke dodaje se trenutni button- u for petlji kako bismo mogli pristupati putem indeksa.

```
370 selectedTrackStyle = lv.style_t()
371 selectedTrackStyle.init()
372
373 selectedTrackStyle.set_bg_color(lv.color_hex(0x7E57C2))
374 selectedTrackStyle.set_bg_grad_color(lv.color_hex(0x5E35B1))
375 selectedTrackStyle.set_bg_grad_dir(lv.GRAD_DIR.HOR)
376 selectedTrackStyle.set_border_color(lv.color_hex(0xE0F7FA))
377 selectedTrackStyle.set_border_width(1)
378 selectedTrackStyle.set_border_side(lv.BORDER_SIDE.BOTTOM | lv.BORDER_SIDE.TOP )
379 selectedTrackStyle.set_text_color(lv.color_hex(0xFFFFFFFF))
380
381 notastyle = lv.style_t()
382 notastyle.init()
383 notastyle.set_bg_opa(lv.OPA._30)
384
385 tipke = []
386
387 for btn_counter in range(7):
388     btn = list.add_button(lv.SYMBOL.AUDIO, "")
389     zadnja_prikazana_pjesma = min(6, len(songList)-1)
390     if btn_counter < len(songList):
391         list.set_button_text(btn, songList[btn_counter][0])
392         tipke.append(btn)
393     if btn_counter == 0:
394         btn.add_style(selectedTrackStyle, 0)
395     else:
396         btn.add_style(trackStyle, 0)
397     if btn_counter == 0:
398         currentButton = btn
```

3 | Raspberry Pi Pico 2

Na naš drugi Raspberry Pi Pico su povezani čitač SD kartice i kolo preko kojeg se emituje signal na 3.5 mm stereo port. On je povezan sa prvim Raspberry Pi Pico-om jumperima na UART0 pinovima, zajedničkim GND, te 2 jumpera preko kojih prvi Raspberry javlja da pristiže određeni signal preko UARTA.

Njegov zadatak je da osluškuje zahtjeve prvog Raspberry-ja, da mu po potrebi posalje listu naziva pjesama i njihovih vremenskih trajanja, te da generiše signale za produciranje zvuka, odnosno pjesama.

U glavni program smo uključili sve potrebne biblioteke i fajlove, i uradili inicijalizaciju pinova.

```
1  import uos
2  import SDCard
3  from wavePlayer import wavePlayer
4  # mount SDCard
5  from machine import SPI,Pin,UART,Timer
6  import time
7  import asyncio
8  import machine
9  import _thread
10 import gc
11 import wave
12 print(machine.freq())
13 #machine.freq(200000000)
14 #print(machine.freq())
15 spi1 = SPI(1, baudrate=5_000_000, sck=Pin(10), mosi=Pin(11), miso=Pin(12))
16 sd = SDCard.SDCard(spi1,Pin(13))
17
```

3.1 | Postavljanje SD kartice

Koristimo biblioteku SDCard (<https://github.com/micropython/micropython-lib/blob/master/micropython/drivers/storage/sdcard/sdcard.py>) za komunikaciju sa SD karticom pomoću SPI. Nakon što deklariramo objekat, izvršimo mount tako da naš Raspberry posmatra memoriju SD kartice kao svoju koja se nalazi u folderu "/sd".

3.2 | Pronalazak pjesama na kartici

Pri pokretanju se formira niz sa path-ovima do pjesama. Za to koristimo listdir funkciju iz biblioteke uos (os), gdje za svaki element provjeravamo da li ima sufiks ".wav", te formiramo listu.

U liniji koda 27, definirana je instanca Player klase wavePlayer. Njena metoda play je glavna funkcija za reprodukciju WAV datoteka u našem programu, a metode pausesong, continueSong i stop su za upravljanje reprodukcijom. Metode play i continueSong će se pozivati u novom thread-u.

```
24 uos.mount(sd, "/sd")
25 pjesma = 0
26
27 player = wavePlayer()
28
29 waveFolder= "/sd"
30 wavelist = []
31
32 for i in uos.listdir(waveFolder):
33     if i.find(".wav")>=0:
34         wavelist.append(waveFolder+"/"+i)
35     elif i.find(".WAV")>=0:
36         wavelist.append(waveFolder+"/"+i)
```

3.3 | Opsluživanje zahtjeva

Postavili smo interrupte na pinove koje koristimo za signaliziranje da pristigla poruka na UART. Jedan pin se koristi za signalizaciju zahtjeva za listom pjesama, te pauziranje i zaustavljanje pjesme. Drugi pin signalizira zahtjeve vezane za pokretanja pjesme.

Nakon što interrupt uzrokuje poziv funkcije, provjeravamo koja je poruka pristigla na UART-u. Ukoliko je poruka "Hello" pokrećemo slanje liste naziva pjesama. Prije slanja liste, šaljemo "start", a po završetku "end". Između, šaljemo naziv pjesme koji smo izdvojili iz path-a, na koji nadovezujemo dužinu iste. Dužinu dobijemo tako što otvorimo svaku pjesmu koristeći biblioteku wave (<https://github.com/joeky888/awesome-micropython-lib/tree/master/Audio>), te izračunamo na osnovu broja frame-ova i framerate-a. Za signale "Play", "Resume", "Stop" i "Pause" pozivamo respektivno funkcije player.play, player.continueSong, player.stop i player.pauseSong, pri čemu je player objekat tipa wavePlayer (<https://github.com/danjperron/PicoAudioPWM>). Ta biblioteka obavlja čitanje podataka iz memorije i njihovo direktno i efikasno prosljeđivanje u registre zadužene za PWM izlaze. Bitno je naglasiti da smo uveli izmjene unutar wavePlayer biblioteke koje omogućuju pauziranje pjesme, kao i nastavljjanje emitovanja pauziranih pjesama od mjesta pauziranja.



```
74 def check_uart(pin):
75     global wavelist, player, timer, pjesma, pokrenuto
76     print("checkkkkkk")
77     if uart1.any():
78         message = uart1.read().decode('utf-8')
79         print('Received:', message)
80         time.sleep(0.1)
81         if message == "Hello":
82             uart1.write("start")
83             time.sleep(1)
84             for i in wavelist:
85                 f = wave.open(i, 'rb')
86                 rate = f.getframerate()
87                 frameCount = f.getnframes()
88                 vrijeme = frameCount/rate
89                 minute = vrijeme//60
90                 sekunde = str(int(vrijeme - minute * 60))
91                 if len(sekunde) < 2:
92                     sekunde = "0" + sekunde
93                 uart1.write(i[4:-4]+" "+str(int(minute))+" "+sekunde)
94                 time.sleep(0.2)
95                 gc.collect()
96             gc.collect()
97             uart1.write("end")
98             time.sleep(0.1)
99
100         elif message == "Stop":
101             print("Stopaga")
102             ugasiPjesmu(0)
103             time.sleep(0.1)
104         elif message == "Pause":
105             print("Pauziramo")
106             player.pauseSong()
```